Reflection Intelligence Manual

RI 反思系統教材 by Danny Ko

📘 前言（**Preface**）

- RI 是什麼
- 目的與應用
- 為何需要反思推論
- 教材使用方式

📘 **Chapter 1：RI** 推理流程總覽

1.1 RI 的理念：讓推論收斂
1.2 反思流程：IO_1 → RL → RS → AL → AO → ReL → IO_2
1.3 反思循環（Iteration）與收斂判斷
1.4 各層的職責與禁止事項
1.5 RI 的優勢與限制

📘 **Chapter 2：RL（Recognition Layer）**問題辨識層

2.1 RL 的角色
2.2 RL 的限制（不能分類、不能修正）
2.3 RS（Reflection Statement）的格式
2.4 如何寫良好的 RS
2.5 RL 常見錯誤
2.6 RL 範例與練習題

📘 **Chapter 3：AL（Analysis Layer）**錯誤分析層

3.1 AL 的目的
3.2 AL 與 RL 的差異
3.3 如何分析 RS 中的錯誤
3.4 EST 與 AL 的關係
3.5 AL 的語氣、深度、界限
3.6 AL 範例練習（基礎 → 進階）

📘 **Chapter 4：EST（Error Structure Taxonomy）錯誤分類系統**
4.1 EST 在 RI 的定位
4.2 六大錯誤類別正式定義
4.2.1 LE
4.2.2 AE
4.2.3 FE
4.2.4 SE
4.2.5 RSE
4.2.6 SCE

4.3 六大錯誤比較表
4.4 錯誤分類流程圖
4.5 每類錯誤的典型案例（至少 3–5 個）
4.6 常見混淆與判斷技巧（例如 AE vs FE）

📘 **Chapter 5：AO（Analysis Output）修正指令層**
5.1 AO 的角色
5.2 AO 四欄格式（type / severity / description / recommended_fix）
5.3 severity 的規則
5.4 description 的語氣與邊界
5.5 recommended_fix 的限制
5.6 AO 的正確寫法與錯誤寫法
5.7 各類錯誤的 AO 實例
5.8 AO 練習題

📘 **Chapter 6：ReL（Refinement Layer）推理修正層**
6.1 ReL 的任務
6.2 ReL 的修正原則
6.3 ReL 能做什麼？不能做什麼？
6.4 ReL 與 AO 的互動方式
6.5 ReL 的常見錯誤（過度修正、偏離題意）
6.6 ReL 實際案例：修正前後對照
6.7 進階 ReL：多輪反思協作

📘 **Chapter 7：IO 層（IO_1 / IO_2）**
7.1 IO_1 是什麼：原始推論
7.2 IO_2 是什麼：修正推論
7.3 $IO_1 \rightarrow IO_2$ 的轉換規則
7.4 何時需要第二輪反思？
7.5 反思收斂（Convergence）標準
7.6 完整示例：$IO_1 \rightarrow RL \rightarrow AL \rightarrow AO \rightarrow ReL \rightarrow IO_2$

📘 **Appendix A：EST 全量案例庫（高階）**
- Appendix A 是什麼？
- A.1 Appendix A 在整本教材中的定位
- A.2 Appendix A 為什麼一定要存在？
- A.3 Appendix A 的結構（標準格式）
- A.4 Appendix A 範例
- A.5 Appendix A 與 RIG 的關係

📘 **Appendix B：FAQ（常見問題）**
- Appendix B 的定位說明
- B.1 AE 與 FE 要如何區分？
- B.2 單純陳述一件事情，會不會算 FE？
- B.3 SE 與 RSE 很像，要怎麼分？
- B.4 AO 可以直接修句子嗎？
- B.5 為什麼 AO 要有 severity？
- B.6 ReL 可以順便「優化內容」嗎？
- B.7 什麼時候需要進入第二輪反思？
- B.8 RIG 在哪裡發揮作用？
- B.9 RI 是不是在模仿人類思考？
- B.10 我可以只用 RI 的一部分嗎？

📘 **Appendix C：RI 專有名詞 Glossary（含 RIG）**
- Appendix C 的定位說明
- C.1 Reflection Intelligence（RI）
- C.2 IO（Inference Output）
- C.3 RL（Recognition Layer）
- C.4 RS（Reflection Statement）
- C.5 AL（Analysis Layer）
- C.6 EST（Error Structure Taxonomy）
- C.7 AO（Analysis Output）
- C.8 ReL（Refinement Layer）
- C.9 Convergence
- C.10 RIG（Reflection Intelligence Grammar）
- C.11 關鍵術語對照

📘 **Appendix D：RI 心智模型（Mindset）**
- D.1 RI 的核心精神：讓推論變得可控
- D.2 RI 與「更聰明」模型的根本差異
- D.3 反思 ≠ 無限思考
- D.4 Convergence 作為核心價值
- D.5 RI 與人類心智的關係
- D.6 SCE：不可逾越的邊界
- D.7 RI 作為「機制」，而非「意識」
- D.8 RI 的長期意義
- D.9 Appendix D 總結

📘 前言（Preface）

什麼是 **RI**（**Reflection Intelligence**）？

Reflection Intelligence（RI，反思智能）是一種讓推論更穩定、可控且可收斂的推理技術。它的核心精神不是「讓模型變聰明」，而是：

讓模型能夠思考自己的思考（**think about its own thinking**）

藉由分層的反思架構（RL → AL → AO → ReL），
RI 將原本一次性完成的推論拆解成多個可監控的步驟，使推論結果更可靠、更一致。

RI 的目的與應用

RI 的主要目的：

- 減少推論幻覺（hallucination）
- 讓錯誤可被發現、拆解、修正
- 讓推論變得可「迭代」而非一次到位
- 增強推論透明度與可解釋性
- 提升邏輯品質與內容完整度

應用情境包含：

- 推理題與邏輯分析
- 專業內容產生（法律、醫學、工程）
- 複雜文本生成
- AI 自我評估 / 自我修正
- 教育與訓練（模型與人類皆適用）

RI 不只是「反省錯誤」，更是一套讓推論過程能被清楚控制的完整框架。

為何需要反思推論？

單段推論常見的問題包括：

- 過度外推（SE）
- 使用未授權知識（AE）
- 不可靠事實（FE）
- 推論跳步（RSE）
- 形式錯誤（LE）
- 安全問題（SCE）

而這些錯誤若沒有被拆解，會累積成更大的幻覺或邏輯不一致。

RI 的反思流程提供以下能力:

- RL:找出問題
- AL:分析問題
- AO:指定修正方式
- ReL:執行精準修正

讓每一次推論都能朝「更正確、更合理、更安全」的方向收斂。

教材使用方式

這份教材將依序介紹:

1. RI 全流程概念(Chapter 1)
2. 每一層的任務與限制(Chapter 2–4)
3. 錯誤分類系統 EST(核心)
4. 如何撰寫 AO 修正指令(Chapter 5)
5. ReL 如何依 AO 修正文句(Chapter 6)
6. $IO_1 \rightarrow IO_2$ 的最終收斂原理(Chapter 7)
7. 附錄案例庫、FAQ、術語、心智模型

你可以:

- 從頭開始學習(建議新手)
- 跳到某個章節補強能力
- 用附錄的案例作為每日訓練材料

本教材的最終目標:

讓你能完全掌握 RI 的結構化推理方法, 並能自行分析、分類、修正、驗證推論品質。

📘 Chapter 1：RI 推理流程總覽

1.1 RI 的理念：
讓推論收斂（Convergent Reasoning）

Reflection Intelligence（RI）是一種讓推論變得可控、可檢查、可修正的智慧系統。
其核心目標並非一次得出正確答案，而是：
透過分層反思，使推論從雜亂 → 清楚、從錯誤 → 正確、從發散 → 收斂。
在傳統推論中，模型一次完成所有思考，容易造成：

- 推理跳步
- 幻覺（hallucination）
- 衝突或矛盾
- 無法修正的錯誤

RI 則透過多層機制拆解推論，使每一層專注於一種任務，
從而建立 穩定、透明、可驗證 的推論框架。

1.2 RI 反思流程總覽：

IO_1 → RL → RS → AL → AO → ReL → IO_2**

這是 RI 的核心流程。

IO_1 = 初始推論（原始答案）
RL = 問題辨識
RS = 問題列表（由 RL 產生）
AL = 錯誤分析
AO = 修正指令
ReL = 根據 AO 精準修正推論
IO_2 = 修正後推論（新版本）

流程運作如下：

1. **IO_1**：模型先給出原始推論。
2. **RL**：只負責找出問題。
3. **RS**：列出全部問題（不分析、不修正）。
4. **AL**：分析每個問題的本質、分類錯誤。
5. **AO**：針對每個錯誤，提出修正建議。
6. **ReL**：依 AO 修正文句，使結果提升品質。
7. **IO_2**：輸出修正後的最終答。

1.3 反思循環（Iteration）與收斂判斷

RI 的強大之處在於「反覆」：

IO_2 → RL → AL → AO → ReL → 下一版 IO

如果：

- 還有錯誤 → 進入下一輪
- 沒有新錯誤 → 收斂，停止反思

收斂代表：

推論已達到合理、穩定、無重大錯誤的狀態。
反思循環可以是 1 輪，也可以是 2～3 輪。
重點不是次數，而是「品質提升」。

1.4 各層的職責與禁止事項（非常重要）

RI 能成功的原因，是每一層只做自己的事，絕不越界。

層級	職責	禁止事項
RL	找問題	❌ 不分析、❌ 不分類、❌ 不修正
AL	分析錯誤、分類錯誤（EST）	❌ 不修正文句
AO	依分類制定修正指令	❌ 不直接寫新句子
ReL	根據 AO 修正句子	❌ 不新增題外內容、❌ 不外推
IO_1 / IO_2	提供推論前後版本	❌ 不自我解釋

這樣的分工讓反思流程不混亂、不跳步、不自我干擾。

1.5 RI 的優勢與限制

⭐ **RI 的優勢**

- 減少幻覺（hallucination）
- 推論更精準且可控
- 內容更一致、不矛盾
- 錯誤可被清楚定位
- 修正有明確邏輯，不靠直覺
- 非常適合專業領域推論（法律、數據、科學等）

⚠️ **RI 的限制**

- 執行時間比一次推論更長
- 需要遵守層級規則，不能亂跳
- AO 與 ReL 若越界，會產生混亂
- 如果 RS 遺漏錯誤，下輪才會被補出

RI 的精神不是讓模型變得更「天才」，而是讓推論變得更「穩定、可靠」。

📘 Chapter 2：RL（Recognition Layer）問題辨識層

2.1 RL 的角色（Role of Recognition Layer）

RL（Recognition Layer）是 RI 反思流程中的第一個“反思層”。
它的唯一任務是：

> 辨識問題，而不是處理問題。

RL 不需要理解推論的深度、不需要判斷錯誤分類、不需要提出修正，
只需要做一件事：

> 告訴 **AL**：原推論（**IO_1**）中有哪些地方可能需要修正。

RL 是整個反思系統的起點，它的品質決定反思是否能順利進行。
如果 RL 沒有找到錯誤，AL 就無法處理，AO 也無從下手。

2.2 RL 的限制（非常重要！）

RL 絕對不能越界。

以下是 RL 的“禁止事項”：

RL 不能做什麼	原因
❌ 不能分析錯誤本質	這是 AL 的工作
❌ 不能分類錯誤（**FE / AE / SE** 等）	這是 EST 與 AL 的職責
❌ 不能提出修正方法	這是 AO 的工作
❌ 不能改寫句子	這是 ReL 的工作
❌ 不能給建議或強度判斷	RL 是中立的觀察者
❌ 不能做推理延伸或猜測	RL 只看原句，不做推論

因此 **RL** 的功能非常純粹，只做：列出問題，不做任何推論處理。

2.3 RS（Reflection Statement）是 RL 的輸出格式

RL 的成果是一份「問題清單」，稱為 RS。
其格式為：

RS:
1. 可能存在 X 問題。
2. Y 的描述不夠清楚。
3. Z 的前提可能不明確。

RS 的語氣要：

- 中性（不下判斷）
- 模糊但明確方向（指出問題但不解釋原因）
- 不做分類、不給定錯誤類型

RS 的目的：
提供 AL 可用的「候選問題列表」。
AL 會根據這些項目分析其本質。

2.4 如何寫良好的 RS

一個好的 *RS* 具備：

✔ 1. 精準指出句子的可疑區域
例如：
- 「關於因果關係的部分可能需要檢查。」
- 「這裡的概念連結可能不足。」

✔ 2. 不加入推測、不加入世界知識
RL 不需要知道事實真不真，只看“是否有可能有問題”。

✔ 3. 不解釋原因、不判斷錯誤類型
這是 AL 的工作。

✔ 4. 用條列清單形式列出所有可能問題
RS 通常包含 2～5 個問題點。

✔ 5. 語氣不中斷、不批判、不確定性語氣較高
例如：
- 可能
- 似乎
- 可能需要釐清
- 前提可能不足

2.5 RL 常見錯誤（需避免）

❌ 錯誤 1：開始分類錯誤（AE、FE、SE 等）

這是 AL 的職責。

❌ 錯誤 2：開始提出修正（like AO）

RL 不能說：

-「應該把句子改成…」

-「建議補充…」

❌ 錯誤 3：開始改寫句子（ReL）

❌ 錯誤 4：評論正確性（事實對錯）

RL 不判斷真偽。

❌ 錯誤 5：語氣太肯定或太武斷

不應該說：

-「這句話有錯。」

-「這裡一定是錯誤。」

而應該說：

-「這裡可能存在推論不足。」

-「這裡可能需要釐清。」

2.6 RL 範例與練習題

範例句子：

「海豚會微笑，因此它比多數動物聰明。」

RL 產生的 *RS* 示例：

RS:

1. 微笑與智力之間的關係可能需要釐清。
2. “多數動物” 的比較範圍可能不明確。
3. 結論的支持度可能需要檢視。

你會看到：

- RL 沒有說這是 SE
- 也沒說這是 FE
- 也沒有做比較
- 完全只是指出「可能問題」

📘 Chapter 3：AL（Analysis Layer）錯誤分析層

3.1 AL 的目的（Purpose of Analysis Layer）

AL 是 RI 反思流程中負責「分析與分類錯誤」的核心層。

如果 RL 是「找出問題」，那麼 AL 的任務就是：

> 理解這些問題的本質，並依照 **EST**（六大錯誤分類）進行分類。

AL 不修正句子，也不提出建議，它只專注於分析，回答最重要的問題：

> 這個句子到底錯在哪裡？屬於哪一類錯誤？

AL 的品質會直接決定 AO 的品質。
分類越準確 → AO 指令越精準 → ReL 修正效果越好。

3.2 AL 與 RL 的差異（極重要）

功能	**RL**（找問題）	**AL**（分析問題）
任務	找出“可能問題”	分析問題本質
輸出	RS（問題清單）	EST 類型分類
是否分類錯誤	❌ 不能	✔ 必須
是否修正句子	❌ 不能	❌ 不能
是否提出建議	❌ 不能	❌ 不能
是否看原因	❌ 不看	✔ 深入分析

一句話總結：

> **RL** 找方向，**AL** 做診斷。

如果 *RL* 是護士做初步檢查，那 *AL* 就是醫生下診斷，而 *AO* 是開處方，*ReL* 是治療。

3.3 如何分析 RS 中的錯誤（AL 的思考方法）

AL 會針對 RS 的每一項描述，進行如下操作：

✔ 第一步：理解句子的意義與邏輯

AL 必須清楚知道句子的推論鏈是否成立。

✔ 第二步：找出錯誤的本質

這裡 AL 會問：

- 前提是否可靠？（FE）
- 是否加入未授權規則？（AE）
- 結論是否外推？（SE）
- 推論鏈是否缺步驟？（RSE）
- 形式是否成立？（LE）
- 是否安全性問題？（SCE）

✔ 第三步：分類錯誤（使用 **EST**）

每個 RS 項目都應對應到：

- 一個或多個 EST 類型
- 並判斷哪個最核心（primary type）

✔ 第四步：以中立語氣描述錯誤

AL 的描述格式通常如下：

```
The statement appears to rely on an unsupported assumption, as
it implies that...
```

或

```
The reasoning jumps from A to C without providing the
intermediate justification...
```

語氣必須專業、中立、不評論內容好壞。

3.4 EST 與 AL 的關係（分類標準來源）

AL 無法脫離 EST 運作。
它的全部分析框架，完全建立於六大錯誤：

- **LE**：邏輯形式錯誤
- **AE**：未授權推論
- **FE**：事實缺乏可靠性
- **SE**：外推過度
- **RSE**：推論鏈缺口
- **SCE**：危險／敏感錯誤

AL 的工作流程是：

1. RL 找出問題
2. AL 將問題套入 EST 結構分析
3. AO 根據分類寫修正指令

因此：

💡 **AL 是 “EST 的使用者”**
💡 **AO 是 “EST 的執行者”**

EST 是 AL 存活的基礎。

3.5 AL 的語氣、深度、界限（Boundaries）

AL 的語氣須遵守以下規範：

✔ 中立

不誇張，不批判，不斷言。

✔ 描述錯誤，而非重寫句子

AL 絕不能改寫句子，也不能說怎麼修正（那是 AO 的任務）。

✔ 分析本質，而非表面

AL 要做的是理解推論的邏輯，而不是看到問題字面就分類。

✔ 不使用情感類詞語

例如：
❌ 糟糕的推論
❌ 很奇怪
❌ 這句話太過份

改成：
✔ The reasoning may rely on…
✔ The connection between A and B appears insufficient.

✔ 不進行 **ReL** 式調整

AL 不會說：
❌ “應該加強因果關係”
❌ “應該刪掉前半句”

這些會出現在 AO。

3.6 AL 範例練習（基礎 → 進階）

句子例子：

「海豚會微笑，因此牠比多數動物聰明。」

RS（由 **RL** 產生）：

1. 微笑與智力的關係可能不明確。
2. “多數動物” 的比較範圍可能不清楚。
3. 結論的支持度可能需要檢視。

AL 對 **RS** 的分析：

```
AL1:
type: SE
description: The conclusion overextends the strength of the
evidence, as the premise does not justify a statement about
intelligence relative to most animals.

AL2:
type: RSE
description: The reasoning jumps from “smiling” to
“intelligence” without establishing the causal or conceptual
connection.

AL3:
type: FE
description: The statement relies on an unverified assumption
that smiling indicates high intelligence.
```

你可以看到：

- AL 清楚分類
- AL 描述錯誤本質
- AL 不修正句子
- AL 使用 EST 類型

這就是完美的 AL。

📘 Chapter 4：EST（Error Structure Taxonomy）錯誤分類系統

4.1 EST 在 RI 的定位

EST（Error Structure Taxonomy）是 RI 系統中用來分類推論錯誤的核心架構。
它由六大錯誤構成，分別代表推論中不同的失誤來源。
在 RI 的反思流程中：

- RL 找出可能的問題
- AL 使用 EST 來分類問題
- AO 根據 EST 類型制定修正方法
- ReL 根據 AO 修正推論

因此：

> **EST 是 AL 運作的基礎，也是 AO 資訊的來源。**
> **沒有 EST，就無法讓 RI 進行精準分析與修正。**

EST 的六大類型各自代表不同的推論缺陷，本章將完整介紹它們的定義、範例及判斷方式。

4.2 六大錯誤類別正式定義

4.2.1 LE（Logical Error）邏輯形式錯誤

定義

推論的「形式」不成立，也就是 前提 → 結論的邏輯結構無效。

常見類型

- 倒置推論（If A then B ≠ If B then A）
- 充分條件 vs 必要條件混淆
- 前提矛盾
- 偷換定義（equivocation）
- 不當的邏輯結構

例子

「人會走路，狗也會走路 → 所以狗是人。」（定義偷換）

4.2.2 AE（Assumption Error）未授權假設錯誤

定義

推論過程中加入了 題目未提供的規則或因果關係。

特徵

- 不是真偽的問題
- 而是「你不能用題目沒有給你的東西」

例子

- 題目沒說草食動物不攻擊 → 卻加入推論
- 題目沒說夜行性 → 視力好 → 卻加入推論

4.2.3 FE（Fact Error）事實錯誤 / 不可靠的世界知識

定義

使用了 不可靠、不普遍、未證實或錯誤的世界知識 當成推論的基礎。

FE 不要求“完全錯”，而是：

不足以當成推論依據。

特徵

- 不普遍
- 未證實
- 過度簡化
- 偽常識（pseudo-common-sense）

例子

- 「草食動物都很溫和。」
- 「老人一定健忘。」

4.2.4 SE（Scope Error）範疇外推錯誤

定義

將結論推得 超出前提可支持的範圍。

典型特徵

- 可能 → 一定
- 有些 → 全部
- 常見 → 必然如此

例子

「有些學生很努力 → 所以所有學生都努力。」

4.2.5 RSE（Reasoning Structure Error）推理結構缺失

定義

推論鏈 A → B → C 中缺少關鍵步驟，導致 跳推論（jumping）。

特徵

- 跳步
- 缺理由
- 中間邏輯橋接不完整

例子

「牠跑得快 → 所以牠很聰明。」

4.2.6 SCE（Safety & Compliance Error）安全／合規錯誤

定義

內容或推論涉及危險、違法、偏見、倫理風險，必須立即中止。

例子

- 教人製造炸藥
- 推斷敏感身份
- 自傷建議

4.3 六大錯誤比較表

錯誤類別	本質	何時發生	關注點
LE	邏輯形式無效	推論結構失效	形式能否推出結論？
AE	未授權規則	題目沒提供 A→B	推論是否加入額外假設？
FE	不可靠知識	世界描述不穩定	這是可靠事實嗎？
SE	結論推太遠	語氣過度、範圍外推	結論是否超出前提？
RSE	推論鏈缺步驟	缺連結或過度跳推	A→C 是否缺 B？
SCE	危險內容	安全 / 法律 / 敏感	是否必須立即中止推論？

4.4 錯誤分類流程圖

以下流程協助 AL 快速分類錯誤:

1. 句子是否在描述世界事實？
└→ 若不可靠 → FE

2. 是否使用題目未授權的規則或因果？
└→ 是 → AE

3. 結論是否超過前提可支持的範圍？
└→ 是 → SE

4. 推論鏈是否缺步驟？
└→ 是 → RSE

5. 推論形式本身是否無效？
└→ 是 → LE

6. 是否包含危險或敏感內容？
└→ 是 → SCE

4.5 每類錯誤的典型案例

LE 範例

1. If A then B，因此 If B then A。
2. 人會呼吸 → 狗會呼吸 → 狗是人。
3. 前提自相矛盾的推論。

AE 範例

1. 草食 → 不攻擊（題目沒提供）
2. 夜行性 → 視力好
3. 會笑 → 智商高

FE 範例

1. 老人都健忘。
2. 草食動物都溫和。
3. 勤奮的人一定成功。

SE 範例

1. 有些貓聰明 → 所有貓都聰明。
2. 海豚會微笑 → 智力比多數動物高。

RSE 範例

1. 跑得快 → 聰明。
2. 加班多 → 幸福感高。
3. 會唱歌 → 會作曲。

SCE 範例

1. 如何製作爆裂物？
2. 判斷某人國籍、宗教或敏感背景
3. 任何危險行為誘導

4.6 常見混淆與判斷技巧

AE vs FE（最常混淆）

- **AE**：用了題目沒說的規則（邏輯規則多出來）
- **FE**：用了不可靠的世界知識（事實本身不穩定）

📌 **AE** 的錯：你不能用
📌 **FE** 的錯：你用的根本不是可靠事實

SE vs RSE

- **SE**：推太遠（強度過度）
- **RSE**：推太快（缺步驟）

📌 **SE** 是「過度強化」
📌 **RSE** 是「過度跳躍」

LE vs RSE

- **LE**：形式完全無效
- **RSE**：形式可能有效，但步驟缺少

SCE 判斷原則

只要：

- 可能危險
- 涉及敏感身份
- 涉及違法
- 涉及傷害

立即歸類 SCE，不進行其他分析。

📘 Chapter 5：AO（Analysis Output）修正指令層

5.1 AO 的角色（Role of Analysis Output）

AO（Analysis Output）是 RI 反思流程中的第三個核心層級，
負責將 AL（Analysis Layer）對錯誤的分類與理解，轉化為：
清楚、具方向性、可執行且不越界的修正指令（recommended_fix）
AO 的任務是：

- 不改寫推論
- 不生成新內容
- 不替 ReL 做修正
- 不加入未授權知識
- 不評斷對錯
- 只描述錯誤本質 + 指出修正方向

AO 是整個反思流程中最重要的「控制點」。
AL 找出錯誤；AO 決定如何修；ReL 才能修得準確。

一句話總結：

> **AO 是開處方，ReL 才是治療。**

5.2 AO 的四欄格式（The Four-Column Schema）

AO 的四欄格式 (每條 *AO* 都必須包含四個欄位*)*

欄位	說明	注意事項
type	EST 錯誤類型（LE / AE / FE / SE / RSE / SCE）	不得猜測，必須精準
severity	嚴重程度（Low / Medium / High）	評估錯誤對推論品質的影響
description	錯誤本質描述	中立、不越界、不修句、不提出新資訊
recommended_fix	修正方向（非修正文句）	指令化、方向明確、不得改寫內容

5.3 severity 的規則（如何判定嚴重程度）

等級	何時使用	例子
Low	錯誤輕微，不影響主要推論	小幅語氣偏差、模糊描述
Medium	常態錯誤，需修正才能收斂	SE、AE、RSE 多屬此類
High	推論完全不可用或危險	LE 大錯、SCE、安全風險

判定原則：

- 是否會阻礙推論收斂？（→ Medium 或 High）
- 是否完全破壞推論？（→ High）
- 是否可局部修正？（→ Low 或 Medium）

大多數 EST 錯誤屬 Medium；

SCE 與重大 LE 幾乎一定是 High。

5.4 AO 的語氣與邊界（Tone & Boundaries）

AO 必須保持：

✔ 中立

- 不評論
- 不情緒化
- 不批判語氣

✔ 不越界

AO 不得：

- 改寫句子（這是 ReL 的工作）
- 發明新前提（會變 AE）
- 引入世界知識（可能變 FE）
- 給具體句子（ReL 才能寫新的句子）

✔ 使用指令語氣

recommended_fix 必須以「請……」開頭，並使用：

- 調整
- 降低
- 移除
- 補充
- 重建
- 停止

等動作動詞。

5.5 recommended_fix 的限制（非常重要）

recommended_fix 只能描述：

修正方向（what to fix）

不能描述句子內容（how to write）

📌 正確示例：

- 「請降低結論強度」
- 「請補足推論鏈」
- 「請移除未授權假設」

❌ 錯誤示例（越界）：

- 「請把句子改成：…」
- 「建議加上 ‘因為牠笑所以…’」
  「你應該寫成：…」

5.6 EST → AO recommended_fix 標準模板（核心章節）

以下模板可直接用於所有 *AO*：

🟦 **RSE**（推論鏈缺失）
請補充前提與結論之間的推論步驟，或調整推論使其不依賴缺失的中介理由。

🟦 **SE**（外推過度）
請降低結論的強度，使其與前提能支持的資訊保持一致。

🟦 **AE**（未授權假設）
請移除推論中未被前提授權的假設，或補充必要前提以支持此推論。

🟦 **FE**（不可靠世界知識）
請避免使用不具普遍性或未證實的世界知識，並改以更可靠或可驗證的描述支持推論。

🟦 **LE**（邏輯形式錯誤）
請重建推論的邏輯結構，使前提與結論之間具備有效的推論形式。

🟥 **SCE**（安全／合規錯誤）
此內容涉及安全或合規風險，請停止此方向的推論，並改以符合安全規範的方式處理問題。

5.7 AO 的正確寫法與錯誤寫法

✔ 正確 AO（範例）

```
type: SE
severity: Medium
description: 結論強度超出前提所能支持的範圍。
recommended_fix: 請降低結論的強度，使其與前提能支持的資訊保持一致。
```

❌ 常見錯誤 **AO**

錯誤類型	示例	問題
改寫句子	「請改成：牠可能聰明」	這是 ReL 的任務
新增內容	「請補上牠有高智商」	AE
評論語氣	「這句話邏輯太差」	不中立
過度具體	「請刪掉第二句並換成…」	越界

5.8 AO 練習題（附解析）

原句：「這位學生很害羞，所以他未來一定無法成為領導者。」

RS:

1. 害羞與領導能力之間的關聯性可能需要釐清。
2. 結論中使用「一定」的語氣可能過強。
3. 前提與未來職業能力之間的推論鏈可能不足。

AL（錯誤分類）

AL1（對應 RS1）

- type: AE（未授權假設）
- 理由：假設「害羞 → 不適合領導」為未授權推論規則。

AL2（對應 RS2）

- type: SE（外推過度）
- 理由：結論使用「一定」造成強度超出前提支持度。

AL3（對應 RS3）

- type: RSE（推論鏈缺失）
- 理由：從性格特質直接跳到「未來無法成為領導者」，缺乏推論步驟。

AO（標準範本文答案）

AO1:

type: AE
severity: Medium
description: 推論假設「害羞會導致無法成為領導者」，但此假設並未被前提授權。
recommended_fix: 請移除推論中未被前提授權的假設，或補充必要前提以支持此推論。

AO2:

type: SE
severity: Medium
description: 結論使用「一定無法成為領導者」的語氣可能超出前提所能支持的範圍。
recommended_fix: 請降低結論的強度，使其與前提能支持的資訊保持一致。

AO3:

type: RSE
severity: Medium
description: 前提中的性格描述與結論中的職業能力推論之間缺乏必要的推論步驟。
recommended_fix: 請補充前提與結論之間的推論步驟，或調整推論使其不依賴缺失的

📘 Chapter 6：ReL（Refinement Layer）推理修正層

6.1 ReL 的任務（Role of Refinement Layer）
ReL（Refinement Layer）是 RI 反思流程中唯一「可以動到原始推論文字」的層級。
它的任務不是重新思考問題，而是：
根據 AO（Analysis Output）所給出的修正指令，對推論進行最小且精準的修正。
一句話定義：

ReL = 嚴格執行 AO 指令的修正執行者。

ReL 不做分析、不再分類、不加入新知識，只負責「執行」。

6.2 ReL 的修正原則（Core Principles）

ReL 必須遵守以下原則：

1 最小修改原則（Minimal Change）
只修改 AO 指出需要修正的部分，其餘內容保持不變。

2 不外推原則（No Expansion）
不增加新的推論、不補充題外資訊、不強化結論。

3 不再判斷原則（No Re-analysis）
ReL 不重新判斷對錯，只依 AO 執行。

4 可回溯原則（Traceability）
IO_2 應能清楚對應回 AO 中的每一條修正指令。

6.3 ReL 能做什麼？不能做什麼？

✔ ReL 可以：

- 降低語氣強度（一定 → 可能）
- 移除未授權假設
- 補齊缺失的推論連結（僅限 AO 指示）
- 重整句子結構以符合邏輯

❌ ReL 不能：

- 新增世界知識（避免 FE）
- 創造新因果關係（避免 AE）
- 強化結論（避免 SE）
- 改寫 AO 沒有要求的內容

6.4 ReL 與 AO 的互動方式

ReL 是 AO 的「執行層」，不是合作層。

關係為：
AO 說「修哪裡、往哪個方向修」
ReL 才決定「怎麼動文字」

若 AO 不清楚，ReL 不得自行補完。

6.5 ReL 的常見錯誤

❌ 過度修正（Over-correction）
把小錯修成大改寫。

❌ 引入新資訊
在修正時加入題目未給的內容。

❌ 偷偷分析
在修正時重新判斷邏輯與分類。

6.6 ReL 實際案例：修正前後對照

IO_1：
「這位學生很害羞，所以他未來一定無法成為領導者。」

AO 指令摘要：

- 移除未授權假設（AE）
- 降低結論強度（SE）
- 補足推論鏈（RSE）

IO_2（ReL 執行結果）：
「這位學生較為害羞，這可能會影響他在某些領導情境中的表現。」

6.7 進階 ReL：多輪反思協作

當 IO_2 仍存在問題時，可再次進入 RL → AL → AO → ReL。

多輪 ReL 的目標不是完美，而是逐步收斂。

📘 Chapter 7：IO 層（IO_1 / IO_2）

7.1 IO_1 是什麼：原始推論

IO_1 是系統或模型在沒有反思機制介入下，所產生的原始推論結果。
IO_1 可能包含：

- 隱含假設
- 外推
- 跳步
- 偽常識

RI 的存在目的，就是讓 **IO_1** 不再是最終答案。

7.2 IO_2 是什麼：修正後推論

IO_2 是經過完整 RI 流程後產生的「可接受推論版本」。
IO_2 的特徵：

- 錯誤已被處理或緩解
- 結論強度合理
- 推論結構完整
- 無重大安全風險

7.3 $IO_1 \rightarrow IO_2$ 的轉換規則

轉換必須：

- 完全可回溯（對應 AO）
- 不新增未分析內容
- 維持原題意

7.4 何時需要第二輪反思？

若 IO_2 仍出現：

- 新的 AE / FE / SE
  修正導致的新問題
  收斂未達標

則必須再次進入反思循環。

7.5 反思收斂（Convergence）標準

當滿足以下條件時可視為收斂：

- 無 High severity 錯誤
- Medium 錯誤已處理
- 推論穩定且一致

7.6 完整範例（摘要）

$$IO_1 \rightarrow RL \rightarrow RS \rightarrow AL \rightarrow AO \rightarrow ReL \rightarrow IO_2$$

📘 Appendix A：EST 全量案例庫（高階）

Appendix A 是什麼？

Appendix A 是 **RI** 的「訓練核心資料集（**Training Corpus**）」，
不是理論章節，而是 大量、可反覆使用的標準化案例庫。

一句話定義（可直接放教材）：

> **Appendix A 收錄依照 RI 全流程（RS → EST → AO → ReL → IO_2）完整標註的高品質錯誤案例，用於訓練、校準與驗證推論品質。**

A.1 Appendix A 在整本教材中的定位

項目	說明
類型	Appendix（附錄）
功能	訓練 / 對照 / 校準
角色	不是教你「是什麼」，而是讓你「練到準」
使用時機	Chapter 4–6 之後

👉 **Chapter 1–7 是「系統」，Appendix A 是「肌肉記憶」**

A.2 Appendix A 為什麼一定要存在？

因為 EST、AO、ReL 不是看懂就會，而是：

- AE vs FE 很容易混
- SE vs RSE 常誤判
- AO 很容易越界
- ReL 很容易外推

📌 只有大量「標準答案級案例」，才能讓判斷穩定下來。

A.3 Appendix A 的結構（標準格式）

每一個案例 必須完整包含以下欄位：

```
Case ID
Original Statement (IO1)
RS (by RL)
EST Classification (by AL)
AO (type / severity / description / recommended_fix)
ReL Result (IO2)
Notes (為何不是其他類型)
```

這是 **RI** 的黃金案例格式。

A.4 Appendix A 範例：

Case A-17（RSE）

IO_1（原句）
「這隻狗跑得很快，所以牠一定很聰明。」

RS（RL）

1. 跑得快與聰明之間的關係可能需要釐清。
2. 結論的支持度可能不足。

EST（AL）

- Primary: RSE
- Secondary: SE

AO
type: RSE
severity: Medium
description: 前提與結論之間缺乏必要的推論步驟，從速度直接跳至智力判斷。
recommended_fix: 請補充前提與結論之間的推論步驟，或調整推論使其不依賴缺失的中介理由。

IO_2（ReL）

「這隻狗跑得很快，但僅憑此點無法判斷牠是否聰明。」

Notes:

- 不是 FE：因為問題不在「快不快是否真」
- 不是 AE：沒有新增規則
- 核心錯誤是 跳步

A.5 Appendix A 與 RIG 的關係

Appendix A 其實是 RIG 的實體化證據：

- AO 有沒有越界？
- ReL 有沒有外推？
- RS 有沒有偷分析？

👉 每一個案例，都是在驗證 RIG 是否被遵守。

📘 Appendix B：FAQ（常見問題）

Appendix B 的定位說明

Appendix B 並非補充理論，而是用來解決實際操作 RI 時最常出現的混淆、誤判與越界行為。
如果說：

- Chapter 1–7 是「設計圖」
- Appendix A 是「訓練資料」

那麼 Appendix B 就是「操作手冊中的除錯區（Troubleshooting）」。
本附錄以「問答形式」整理 RI 使用者在學習與實作過程中，最容易犯錯或卡關的關鍵問題。

B.1 AE 與 FE 要如何區分？

Q：AE 和 FE 看起來都像是「不合理的推論」，差別在哪？
A：關鍵不在於「對不對」，而在於「你能不能用」。

- AE（Assumption Error）
  → 使用了「題目沒有授權」的規則或因果關係
  👉 問的是：這個規則從哪來？

- FE（Fact Error）
  → 使用了不可靠、不普遍、未證實的世界知識
  👉 問的是：這件事本身可靠嗎？

📌 判斷口訣：

- **AE 的錯：你不能用**
- **FE 的錯：你用的不是可靠事實**

B.2 單純陳述一件事情，會不會算 FE？

Q：「貓跳得很高」、「花是紅的」這種句子算 FE 嗎？

A：單純描述，不構成推論，就不列為 FE。

FE 只在以下情況成立：

- 該敘述被當成推論前提
- 且其可靠性不足以支持後續結論

📌 重點：

FE 不是「描述錯」，而是「用錯地方」。

B.3 SE 與 RSE 很像，要怎麼分？

Q：SE（外推過度）和 RSE（推論鏈缺失）差在哪？
A：差在「推太遠」還是「推太快」。

- SE（Scope Error）
  - 問題在「結論強度」
  - 常見於：一定、必然、全部
  - 前提是有的，但被用過頭

- RSE（Reasoning Structure Error）
  - 問題在「中間步驟不見了」
  - A → C，中間 B 不存在

📌 判斷口訣：

- SE = 強度過度
- RSE = 結構跳步

B.4 AO 可以直接修句子嗎？

Q：AO 能不能直接告訴 ReL 該怎麼改寫？
A：不可以。

AO 的責任是：
- 描述錯誤本質
- 指出修正方向

AO 不能：
- 提供實際句子
- 指定文字內容
- 發明新前提

📌 一句話規則：
AO 說「修哪裡」，ReL 才能說「怎麼修」。

B.5 為什麼 AO 要有 severity？

Q：既然 AO 不判斷對錯，為什麼還要 severity？
A：severity 不是道德評價，而是工程優先級。

severity 回答的是：
- 這個錯誤是否會阻礙推論收斂？
- 是否必須立刻修正？
- 是否會造成整體推論不可用？

📌 原則：
- Low：不修也不會崩
- Medium：不修會卡住
- High：不修整個推論報廢 / 有風險

B.6 ReL 可以順便「優化內容」嗎？

Q：ReL 在修正時，能不能順便讓句子更好看？
A：不行。

ReL 的唯一任務是：

- 精準執行 AO 的指令

ReL 不能：

- 加新觀點
- 提升文采
- 修「沒被 AO 指到的地方」

📌 **ReL 的原則是：最小修改、只修被要求的部分。**

B.7 什麼時候需要進入第二輪反思？

Q：什麼情況下 IO_2 還要再進 RL？
A：當 IO_2 出現「新錯誤」時。
常見觸發條件：

- 修正引入新的 AE / FE
- 強度降低後仍有 SE
- 修補 RSE 時外推

📌 收斂不是「沒有缺點」，而是：沒有新的結構性錯誤。

B.8 RIG 在哪裡發揮作用？

Q：我一直聽到 RIG，它到底在做什麼？
A：RIG 不是一個步驟，而是一套約束。
RIG 規定：

- 各層能說什麼
- 不能做什麼
- 語氣到哪裡停

📌 如果你在檢查：

- AO 有沒有越界
- ReL 有沒有偷分析
- RL 有沒有下判斷

你其實就在使用 RIG。

B.9 RI 是不是在模仿人類思考？

Q：RI 是不是在讓 AI 變得更像人？
A：不是。
RI 的目標不是「像人」，而是：

- 推論可控
- 錯誤可定位
- 修正可回溯

📌 **RI** 是工程機制，不是意識模型。

B.10 我可以只用 RI 的一部分嗎？

Q：可以只用 EST 或 AO 嗎？
A：可以，但風險會提高。

- 單用 EST → 容易誤判
- 單用 AO → 容易越界
- 少了 RIG → 系統容易崩

📌 **RI** 的強度來自「結構完整」，
而不是某一層特別聰明。

📘 Appendix C：RI 專有名詞 Glossary（含 RIG）

Appendix C 的定位說明

Appendix C 為 Reflection Intelligence（RI）系統的專有名詞與核心概念對照表，目的在於確保所有讀者、使用者與實作者，對 RI 中的術語有一致、不可歧義的理解。

本附錄不進行延伸說明或教學示例，而是提供：

- 穩定定義（stable definitions）
- 層級歸屬（layer ownership）
- 行為邊界（what it can / cannot do）

📌 當術語理解出現歧義時，以 Appendix C 為最終依據。

C.1 Reflection Intelligence（RI）

Reflection Intelligence（RI）

一種用於將推論過程拆解、監控、修正並使其收斂的結構化反思機制。

- RI 不是模型
- RI 不是意識
- RI 不是人格模擬

👉 RI 是一套工程化的推理控制框架。

C.2 IO（Inference Output）

IO_1（Initial Inference Output）
原始推論輸出，未經反思與修正。

- 不可控
- 可能包含多種錯誤
- 是反思流程的起點

IO_2（Refined Inference Output）
經過完整 RI 流程後的修正推論。

- 可控
- 可回溯
- 可進一步檢查是否收斂

C.3 RL（Recognition Layer）

Recognition Layer（問題辨識層）

- 任務：找出「可能有問題的地方」
- 輸出：RS（Reflection Statement）

不能做的事：

- 不分析錯誤
- 不分類錯誤
- 不修正句子
- 不判斷對錯

👉 RL 只負責「指出問題存在」，不處理問題。

C.4 RS（Reflection Statement）

Reflection Statement（反思陳述）

- 由 RL 產生
- 形式為問題清單
- 語氣中立、保留不確定性

RS 的目的在於：
提供 AL 一組「候選問題」，而非結論。

C.5 AL（Analysis Layer）

Analysis Layer（錯誤分析層）

- 任務：分析 RS 中問題的本質
- 工具：EST（錯誤分類系統）

AL 的核心輸出是：

- 錯誤類型判定
- 錯誤本質描述

AL 不能：

- 修正句子
- 提出具體修法
  進行語言優化

C.6 EST（Error Structure Taxonomy）

錯誤結構分類系統

EST 是 RI 用來分類推論錯誤的正式架構，包含六大類：

- LE（Logical Error）
  邏輯形式不成立
- AE（Assumption Error）
  使用未被題目授權的假設或規則
- FE（Fact Error）
  使用不可靠、不普遍或未證實的世界知識作為推論基礎
- SE（Scope Error）
  結論強度或範圍超出前提可支持的程度
- RSE（Reasoning Structure Error）
  推論鏈缺少必要步驟（跳推論）
- SCE（Safety & Compliance Error）
  涉及危險、違法、倫理或合規風險

C.7 AO（Analysis Output）

Analysis Output（修正指令層）
AO 是將 AL 的分析結果轉換為「可執行修正方向」的控制層。

AO 每一條必須包含四個欄位：

1. type（EST 類型）
2. severity（Low / Medium / High）
3. description（錯誤本質）
4. recommended_fix（修正方向）

AO 的限制：

- 不改寫句子
- 不生成新內容
- 不引入新前提

👉 AO 是開處方，而不是治療。

C.8 ReL（Refinement Layer）

Refinement Layer（推理修正層）

- 任務：嚴格依照 AO 指令修正推論
- 產出：IO_2

ReL 的原則：

- 最小修改
- 不外推
- 不再判斷
- 不新增知識

👉 ReL 只執行，不再思考。

C.9 Convergence

指反思循環停止的狀態，其判斷標準為：

- 未出現新的結構性錯誤
- 推論在合理誤差範圍內
- 不再需要進入下一輪反思

📌 收斂 ≠ 完美
📌 收斂 = 可停止

C.10 RIG（Reflection Intelligence Grammar）

Reflection Intelligence Grammar（RI 語法系統）
RIG 是一套用來約束 RI 各層：

- 語氣（Tone）
- 行為（Action）
- 邊界（Boundary）
- 結構（Structure）

的跨層級規範系統。
RIG：

- 不是 Layer
- 不是 Module
- 不是 Error 類型

👉 RIG 是讓 RI 不混層、不崩壞的語法憲章。

C.11 關鍵術語對照

術語	功能摘要
RI	整體反思推理框架
IO_1	原始推論
IO_2	修正後推論
RL	找問題
RS	問題清單
AL	分析與分類
EST	錯誤分類系統
AO	修正指令
ReL	執行修正
RIG	跨層級語法與邊界

📘 Appendix D：RI 心智模型（Mindset）

D.1 RI 的核心精神：讓推論變得可控

Reflection Intelligence（RI）並不是一種「更聰明的思考方式」，
而是一種 讓思考過程可被檢查、修正與停止的機制。
在傳統推論中，無論是人類或 AI，常見的問題是：

- 推論一次完成，過程不可回溯
- 錯誤被包裹在流暢語言中，難以拆解
- 修正往往依賴直覺，而非結構
- 無法判斷「什麼時候該停止思考」

RI 的出發點並不是追求「最聰明的答案」，
而是追求：

> 在有限資源下，得到一個可接受、穩定、可停止的推論結果。
> 這正是「可控（**controllable**）」推論的本質。

D.2 RI 與「更聰明」模型的根本差異

RI 並不嘗試讓模型：

- 學會更多世界知識
- 建立更複雜的內部信念
- 擁有情感或主觀意識

RI 關注的是另一個問題：
當推論發生誤差時，系統是否知道「哪裡出問題」與「該怎麼修」？

傳統強化方向	**RI** 的方向
更大模型	更清楚的層級
更多參數	更嚴格的邊界
更像人類	更可被控制
更會回答	更會修正

因此，**RI** 是一種 結構化的自我檢查機制，而不是意識模擬。

D.3 反思 ≠ 無限思考

在直覺上,「反思」常被誤解為:

- 一直想下去
- 想得越多越好
- 不斷追求完美答案

RI 對反思的定義恰恰相反。
在 RI 中, 反思的目的是 讓推論收斂, 而不是發散。
因此 RI 明確引入:

- 層級邊界(RL / AL / AO / ReL)
- 錯誤分類(EST)
- 修正指令而非自由改寫(AO)
- 收斂判斷(Convergence)

真正成熟的反思系統, 必須知道什麼時候可以停下來。

D.4 Convergence 作為核心價值

在 RI 中,「收斂」是一個非常重要但容易被忽略的概念。
收斂並不代表:

- 完全正確
- 沒有任何誤差
- 絕對客觀

收斂代表的是:

- 沒有新的結構性錯誤被辨識
- 剩餘誤差在可接受範圍內
- 再進行反思, 成本高於收益

換句話說:
收斂是工程上的「足夠好」, 而不是哲學上的「完美」。
這一點使 RI 非常適合:

- 邊緣設備(Edge AI)
- 即時系統
- 資源受限的環境
- 實際應用場景(而非理論最優)

D.5 RI 與人類心智的關係

RI 並不假設機器具有：

- 情感
- 主觀意識
- 價值判斷

RI 模擬的並不是「人類的感受」，
而是「人類在專業場域中檢查錯誤的方式」。
例如：

- 工程師 Debug
- 醫師做 differential diagnosis
- 法律人拆解論證漏洞

這些行為的共通點是：

- 分步
- 可回溯
- 有明確邊界
- 允許停止

RI 正是把這種 專業反思行為結構化、形式化。

D.6 SCE：不可逾越的邊界

RI 明確承認一件事：
並非所有推論都應該被完成。

因此 SCE（Safety & Compliance Error）在 RI 中具有最高優先級。
只要涉及：

- 人身傷害
- 違法行為
- 敏感身份推斷
- 明確倫理風險

反思流程必須 立即停止或轉向安全處理。
這代表 RI 的反思能力不是無限的，
而是被價值與規範明確約束的。

D.7 RI 作為「機制」，而非「意識」

一個關鍵澄清是：

RI 不是一個會思考的機器，而是一個規範思考的機制。
RI 不儲存信念，
RI 不形成自我，
RI 不追求主體性。

RI 的價值在於：

- 降低推論風險
- 提高系統穩定性
- 縮短錯誤修正時間
- 讓 AI 行為更可預期

這使 RI 成為一種 工程級、制度級 的能力，而非哲學主張。

D.8 RI 的長期意義

從長期來看，RI 的價值不在於單一模型，而在於：

- 作為 AI 系統的「反思層」
- 作為人機協作的「中介結構」
- 作為複雜決策系統的「安全閥」

它提供了一種新的可能性：
即使語言理解存在誤差，只要誤差在可控範圍內，系統仍然可以安全運作。
這正是 RI 與現實世界應用接軌的關鍵。

D.9 Appendix D 總結

反思智能（**Reflection Intelligence**）並不是在打造一台會思考的機器，
而是在建立一個知道如何修正自己——以及何時該停止修正的系統。

Reflection Intelligence is not about building a thinking machine,
but about building a system that knows how to correct itself — and when to stop.